Session 20 Teamwork — Look an Issue Up by Its Own Real Key

Context

Work in your team (six teams, each team of four students). Live code added `Project.issueSeq` and a real `Issue` model (with `key`, uniquely constrained per project), migrated it, and built a real transactional route that creates an issue with an atomically-generated key (`WEB-1`, `WEB-2`, ...). Nobody has shown you, live, how to look an issue up by that key — only by its numeric `id`, inside the same transaction that creates it.

Goal

A real, new, read-only route that returns the real issue matching a given project and a given human-readable key — using a real Prisma feature you research yourselves: `findUnique` with a compound-unique input.

What to build

- **`GET /prisma-check/projects/:id/issues/:key`** — returns the real `Issue` row matching that exact `projectId` + `key` combination.
- **Research required:** `Issue` already has `@@unique([projectId, key])`. Find, in the real official Prisma documentation, the exact shape Prisma generates for a `findUnique` call against a compound (multi-field) unique constraint — it is not simply `{ projectId, key }`.
- **No schema change, no new migration.** The constraint already exists; this is a query task only.
- **Written synthesis required:** the exact generated field name you used, and where it comes from; and why `findUnique` (not `findFirst`) is the structurally correct choice here, in terms of what a `findUnique` call actually guarantees about the schema.

Rules

Allowed: everything already shown live, plus everything you discover by reading the real official Prisma documentation on unique identifiers/compound keys. `npx prisma studio` (optional) to browse the real database visually.

Not yet: do not write a new migration, do not add any new field to `Issue` / `Project`. The cross-project concurrency-timing proof is homework's own job, not yours.

Forbidden: do not touch `schema.prisma`. Do not touch any existing Mongoose module, controller, service, or route. Your new route must live only inside `PrismaCheckController`.

Test it (design your own full proof — this is a starting point, not the complete list)

inside api/

BASH

```
npm run start
```

in a second terminal

BASH

```
curl http://localhost:3000/prisma-check/projects/1/issues/WEB-1
curl -X POST http://localhost:3000/prisma-check/projects/1/issues -H "Content-Type:
application/json" -d '{"title":"Teamwork-created issue"}'
curl http://localhost:3000/prisma-check/projects/1/issues/<the key the POST above just
returned>
curl http://localhost:3000/workspaces
```

Run all four — the last one must return exactly what it already returned before today's teamwork even started.

Done when

- `GET /prisma-check/projects/:id/issues/:key` returns the correct real issue, proven with real seeded/created data.
- A fresh issue created through the existing transactional route can immediately be found again through your new route, using the exact key it was given.
- `GET /workspaces` (the pre-existing Mongoose-backed route) is proven, by actually running it, to be completely unaffected.
- Your written synthesis covers both required points above.

Working time

About 24 minutes — researching the exact compound-unique input shape deserves genuine time before guessing at the syntax.